Array is a structured data type.

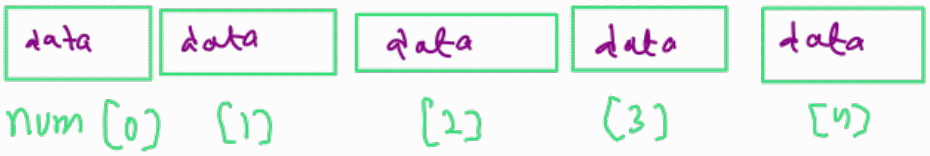
(1-D) array → collection of items with the same data type.

Declaring a array Syntax.

datatype arrayName [int Exp];

→ must be any positive constant that tells us the size of array.

e.g int num [5]



Accessing Array components.

arrayName [Index] → Specifies the position from where the data has to be accessed.

Index starts from 0

Int list [10]; → declared a array called list with (0 → 9) places to hold data

list [5] = 34; → Stores 34 at the position of 0

E.g

Int i = 4

list [2 * i - 3] = 58

2 * 4 - 3 → 5

Stores 58 at position 5.

we can also declare a array as follows.

```
Const Int arraySize = 10;
Int datalist [arraysize];
```

Processing 1-D arrays

```
Int List [100];
for ( int i=0 ; i< 100; i=i+1)
        cin>> list [i];
```

→ Reads data in array.

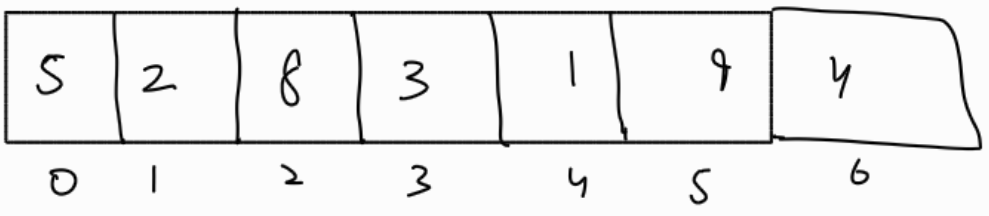
↓

Index running from 0 till 99

Example finding Avg of a array named Sales.

```
double Sales [10]
 double Sum = 0;
  for (int i=0; i<= 9; i=i+1)
       Sum= Sum + Sales [i];
  cout << "Answer is" << Sum /10;
```

finding largest element in a array



Int array [7];

```
Int max = array[0]
```

```
for(int i=0; i<=6; i=i+1)
{
    if(array[i] > max)
        max = array[i];
}
cout << "Max value is" << max;
```

For both max and min Initialise them to first element of array.

Program to Print reverse elements of a array.

Int counter [5]; → Say elements inside are 3, 2, 1, 2, 7

```
for(int J=4; J>=0; J=J-1)
    cout << counter[J] << " ";
```

Array Index out of bounds

```
double num[10];
int i;
```

Meaning num [i]

(i is valid from 0 till Array Size -1)

(Array Initialization)

Index value	0	1	2	3	4

e.g double Sales [5] = {12.2, 13.6, 14.5, 2.6, 75.67}

curly brackets

note: It is not necessary to mention array size during Initialization.

Partial Initialization of Arrays during Declaration

e.g `int list [10] = {0};` Intializes all components of a array to 0.

e.g `int List [10] = { 8, 5, 12 };` declares a array of size 10 and intialses only Index 0 to 8, Index 1 → 5 .... All other components are 0.

Note

`int list [] = { 4, 7 };`

↳ declares / Intializes a array of size 2.

Restrictions → // processing must be done component wise.

copying data from one array to another (of Same Size and datatype)

```
Int mylist [5] = {1, 2, 0, 3, 6};
Int yourlist [5];
for (int i=0;  i<= 4;  i=i+1)
  {
        yourlist [i] = mylist [i];
  }
```

Functions (Arrays are passed by reference only) and hence we don't need & Symbol.

When declaring array as a formal parameter, the size of the array is usually omitted (Ignored by complier.

e.g
```
void funcArrayAsParam ( int listone [], int two[])
{
    .
    .
    .
}
```

two arrays being passed as formal Parameters

However Sometimes no. of elements in a array might be less than its size. Hence we will only process those components of a array that hold the data.

Hence to write function for these we also add a new formal parameter which is size.

```
void Initialise ( int list [], int lise size)
{
    .
    .
    .
}
```

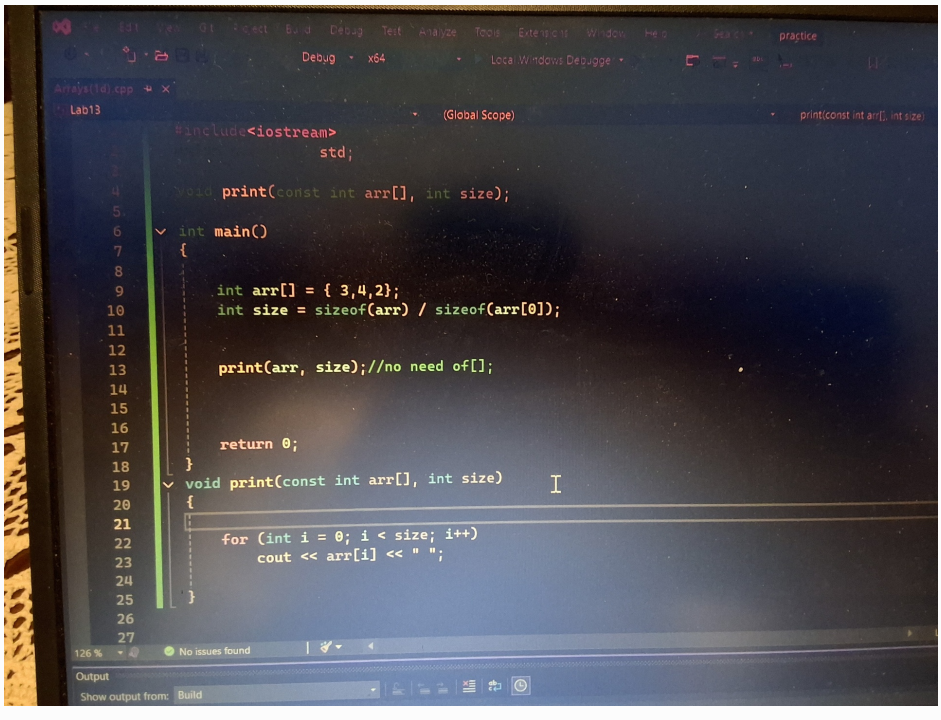
declaration

passed the size of actual array.

Always pass array size seperately in a function.

---

note:: Arrays are passed by ref so any change in a fuction will change the Orignal array in int main(). Hence to avoid this use const.



```
#include<iostream>
using namespace std;

void print(const int arr[], int size);

int main()
{

    int arr[] = { 3,4,2};
    int size = sizeof(arr) / sizeof(arr[0]);


    print(arr, size);//no need of[];


    return 0;
}
void print(const int arr[], int size)
{

    for (int i = 0; i < size; i++)
        cout << arr[i] << " ";

}
```

Constant arrays as formal parameters

when a formal parameter is a reference parameter it also changes the Actual parameter. | Main variable changes

Even though array are passed as by ref you can still prevent the function from changing the array by using

Const

```
Void example (int x [], const int y [], int Sizex)
{
    .
    .
    .
}
```

here in int main the modifiable value is X but not Y array.

To calculate Size of array we use

Int Size :- Sizeof (arrayname) | Size of (arrayname [0]);

e.g

```
void print (int array [], int Size)

Int main ()
{
    Int array [] = {1,2,3,4,5};
        print (array );
                ↳ calling function no need to write []
}
```

Base address of array :: is the address of the first element of the array

```
Int myList [5];
```

if (myList <= List [0]) will evaluate to true as myList gives the address of the array

Functions cannot return a value of any type array

Reversing a 1D array



Rules
→ take the first element and swap it with the last element. take the second element and swap it with the second last element.

```
Int arr [5] = {1, 2, 3, 4, 5}
int temp = 0;

for (int i=0; i< Length /2; i++)

    temp = arr (i);
    arr (i) = arr (length-1-i ]
    arr (length-1-i) = temp;
```

Length is size of array.

dividing by 2 because we are swapping first 2 elements. (3) remains at its position.

e.g

```
Void arrayfunction ( Int array [] , Int listsize)
{
    for (int i=0 ; i< listsize; i++)
        cin>> array [i] << " " ;
}
```

```
Int Sumcalc ( Int array [] , Int listsize)
{
    Int Sum=0;
    for (int i=0; i< list size; i++)
        Sum = Sum + array [i];
    return Sum;
}
```

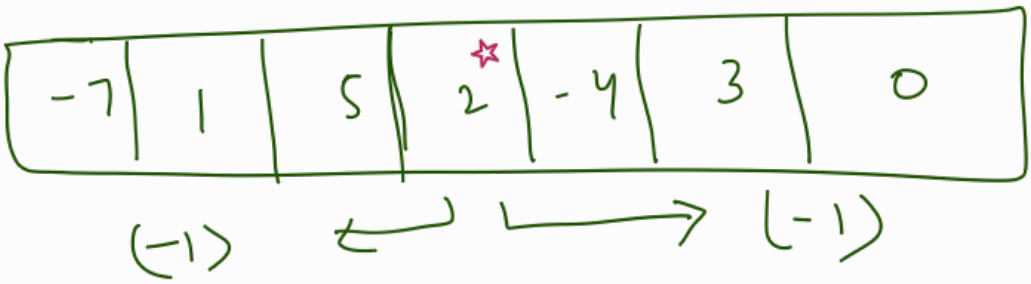
```
Int finder ( Int array [], Int listsize)
{
    Int max=0
    for (int i=0; i< listsize ; i++)
        if (array [i] > max)
            max= array [i];
            temp = i;
    return temp;
}
```

```
Void copy ( Int list1 [], Int size1, Int list2 [], Int size2)
{
    if (size1 == size2)
    {
        for (int i=0; i< size1 ; i++)
```

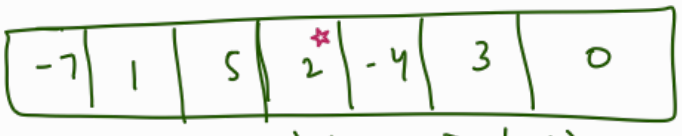
list 2 ← list 1

Finding equilibrium of a array.

Sum of LHS = Sum of RHS



1) cal total sum;
2) right sum = Sum;
3) for each element { transverse complete array
   i) Subtract each element from RS
   ii) compare → if (RS = LS) Break Equilibrium found.
   c) Add to leftsum every element



arr[];

```
int LS=0, RS=0, S=0
for (int i=0; i< 7; i++)
    S= S+ arr[i];
RS= S;
for (int i=0; i< 7; i++)
{
  RS= RS- arr[i];
    if ( RS = LS)
        Break; cout<<i;
  leftSum= leftSum + arr[i];
}
```

Equilibrium if no index is found return -1

Searching an array for specific item / linear Search: done on lapton

Sorting → Ascending
→ descending

Bubble Sort

→ Compare 1st two elements and exchange them if they are out of order.

→ move down One element and compare 2nd and 3rd element

→ Repeating the process over and over again until the list is properly sorted.

Sorted in ASC

Program `int arr[5] = {2, 7, 1, 3, 6};`

```
for(int i=0; i<5; i++)   → goes to use a indexes
{
    for(int J=0; J<Size-1-i; J++)   → goes to 2nd last num in array.
    {
        if (arr[J] > arr[J+1])
        {
```

0	1
7	2

Sorting in Ascending

```
temp = arr[J]
arr[J] = arr[J+1]   → Swapping two numbers.
arr[J+1] = temp
```

7 > 2 ↓ Swap

2 → 7

3 3

Purpose Inner Loop:- To repeatedly compare adjacent elements and swap them if they are at wrong order.

outer Loop:- keeps track of how many passes are needed over the entire array

why use `J < size - 1 - i` for inner loop

Because in the first pass the largest element goes to the Last index.

So at the second pass now as the outer loop value is set to 1. The inner loop would have to make less comparisons / Don't compare whats already sorted.

Selection Sort

→ we search for the min element and replace it with the current index position.

e.g Initially we will first find the smallest element in all of the array and then replace it with the element at index 0.

---

a[] = {6, 5, 3, 2, 1, 4}

1 2 3 4 5
↑
stop the loop here

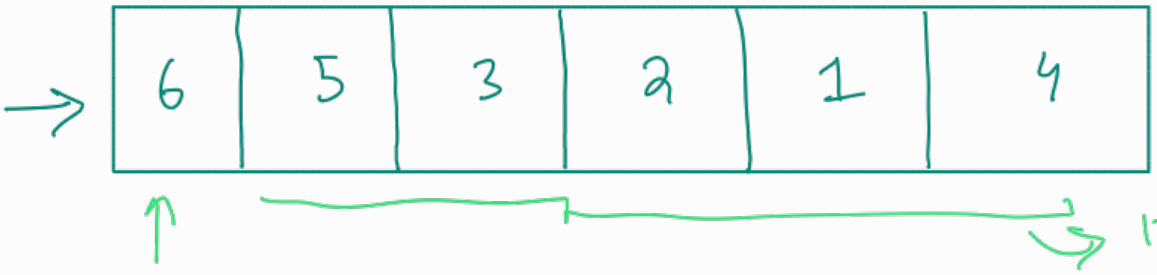
```
for (int i=0 ; i< n-1 ; i++)
{
    int Smallest = i;   → current position.

    for (int j= i+1; j< size; j++)
    {
        if (a[j] < a[smallest])
        {
            Smallest = j;
        }
    }

    if( Smallest != i)
    {
        temp = a[Smallest]
        a[smallest] = a[j]
        a[j] = temp;
    }
```

→ loop for the remaining unsorted array after the Smallest value we had considered.

if they are equal it means array is sorted.



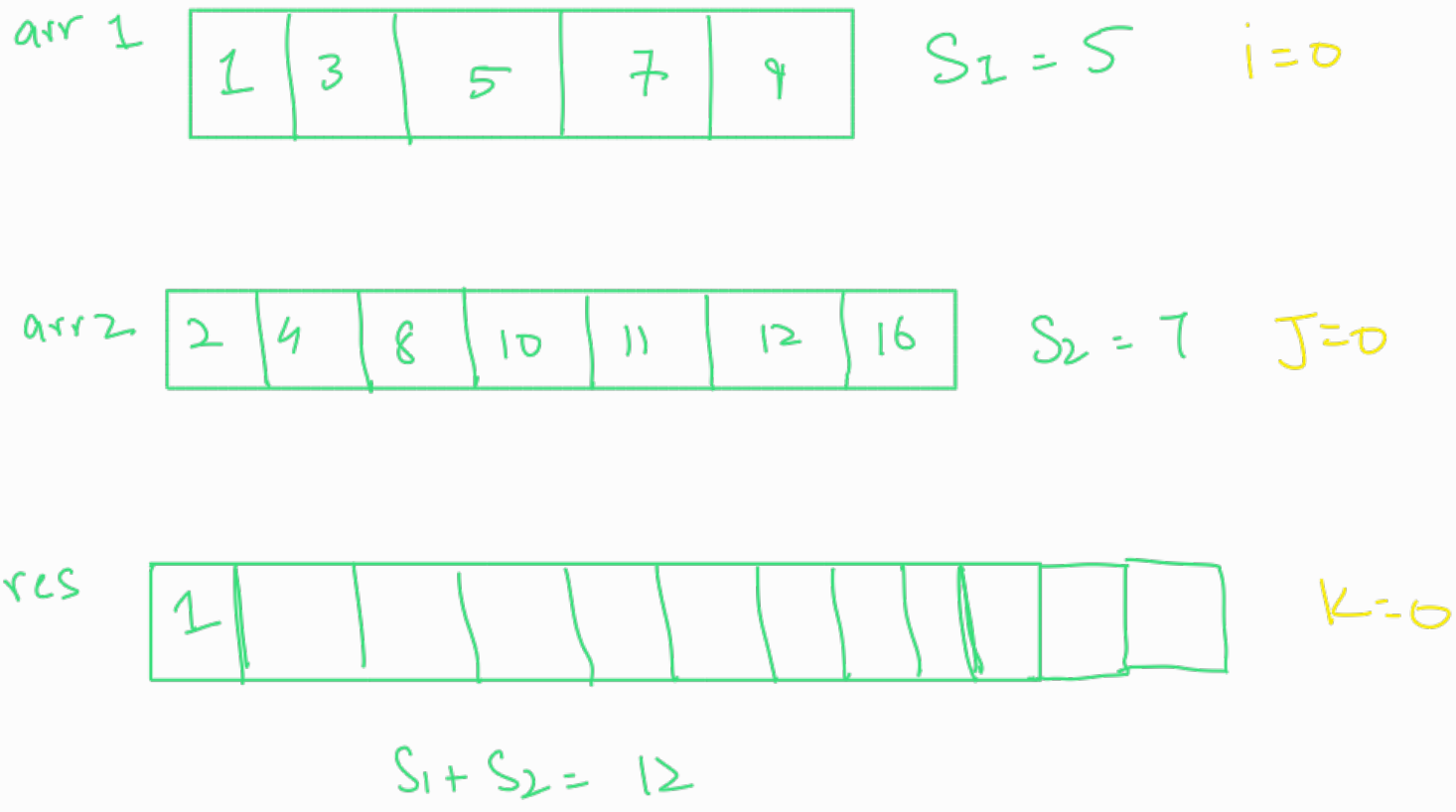
Inner Loop condi: J = i + 1 (for remaining elements)

Int Smallest = considering Index of 6

→ then we will run Loop for rest of the remaining array and compare index of 6 with other Indexes.

And if there is a number Smaller then Sir we will Swap it.

Merge two Sorted arrays into one




```
Int i=0, J=0, k=0
while( i < S1 && J < S2)
  {
    if ( arr1[i] < arr[J] )
       { res [k] = arr1 [i];
         i++, k++;
       }
    else
       { res [k] = arr a [J];
         J++ , k++;
       }
  }
while ( i < S1)
  {
       res [ k ] = arr 1 [ i ];
         k++,  i++;
  }
```

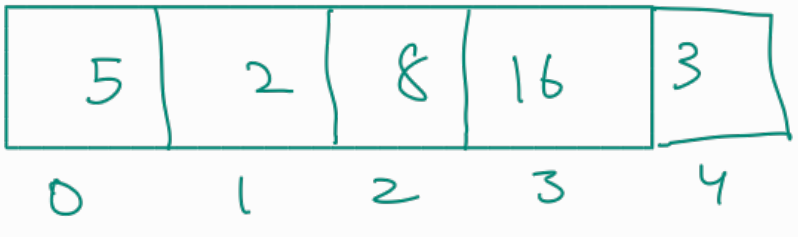
add the remaining elements

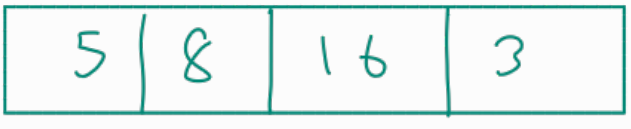
```
while (j < s2)
{
    res[k] = arr2[j];
    k++, j++
}
```

add the remaining elements.

How to delete in a array from a specific position.

example.. user says to delete element from position 1.





So basically overwrite the number which we want to delete with the next number

```
i = i+1;
```

```
for (int i = delelepos; i < size; i++)
    arr[i] = arr[i+1];
```

Loop always start from delete position and then goes till the end overwriting every thing

Now as we have removed one of the variable to print the array we also decrease the size by -1 inorder to Avoid Junck values.

```
for (int i=0; i< Size-1; i++)
        cout<< array[i]<< " ";
```

list of characters

Character array: might not contain a null character.

C-String always contains a null-character.

↳ list of characters that end with an 10.

(Character arrays) → array whose components are of type char.

'\0' → zero

note 'A' → means it is a single character

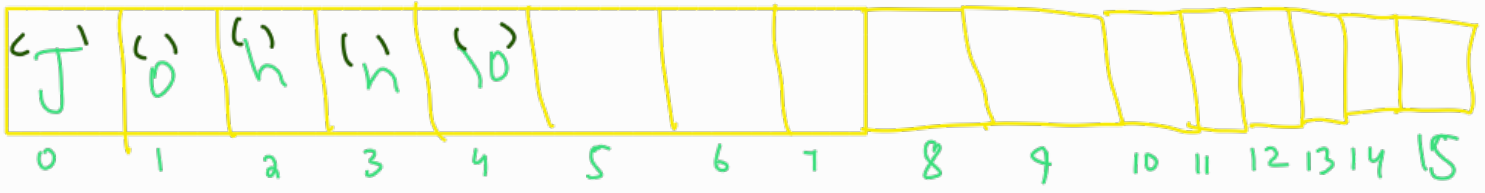
"A" → means it contains two characters 'A' and '\0'.

e.g

Char name [16];

It contains 16 components the largest thing that can be stored is 15.

e.g Storing "John" → Stores 'J' 'o' 'h' 'n' '\0' in Char array



It will use 5 spaces.

Another way to initialise

Char C[]= {'J', 'o', 'h', 'n', '\0'};

now using header files

#Include <Cstring>

Strcpy → just like (=)

Strcmp → compare two string

Strlen → Length of a string gives length excluding "\0".

Strcpy (S1, S2)

[copies data of S2 to S1]

Strcmp (S1, S2) → returns value <0 if S1 is less than S2

=0 if both are same

>0 if S1 is greater than S2.

Strcmp (Bill, Billy) returns <0 as 'l' ascii value is 0 to 'y'.

taking Strings as Inputs

name = furqan
output = furqan

```
char name [31];
cin >> name;
cout << name
```

→ if length of input is 4 the compiler stores 4 in name and lastly a \0.

Inputting Sentences in char arrays.

```
cin.get (str, Size);
```

(name of char array; Size of Sentence)

```
Char name [100];
Cin.get (name, 40)
```

→ make stores 39 words. Then a \0.

Char name [16] = { 'J', 'o', 'h', 'n', '\0' };

Char name [16] = "John"; Same thing

finding Length of a String

```
int len=0;
for(int i=0; name[i] != '\0'; i++)
    len++;
```

final ans = 4

functions for string manipulation → #Include <cstring>

Strcpy (S1, S2) → copies S2 into S1 → length of S2 must be as large as S1

Strcmp (S1, S2) → gives value < 0 if S1 is less than S2.

gives value = 0 if S1 = S2.

gives value > 0 if S1 is greater than S1.

e.g

```
char array [100] = { 'e', 'y', 't', '\0' };
char array1 [100] = { 'J', 'o', '\0' };

Strcpy ( array, array1 );
cout<< array;
```

→ output J o

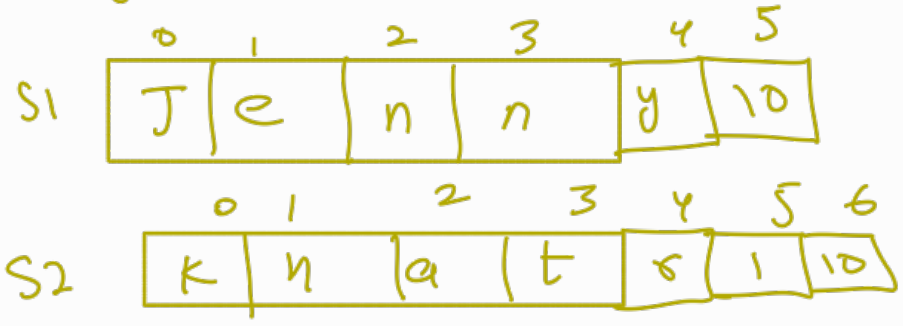
copies all data of array1 to array by over writing

Making string copy function

```
char array[14] = {'J', 'o', 'h', 'n', '\0'}
char array1[14];

int i=0;
for(; array[i] != '\0'; i++)
{
    array1[i] = array[i];
}
array1[i] = '\0';
```

String concatenation



Jenny khatri → combining two strings / concatenation

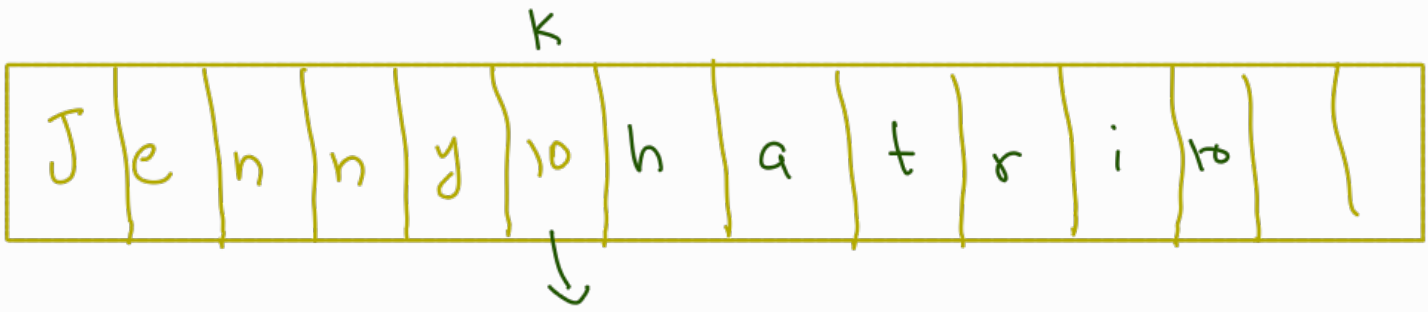
note:: Size of S1 must be large enought to store S2.

Predefined function

strcat(S1, S2) ← concatenates S2 with S1. / joins S2 with S1. final data is stored in S1.

```
char S1 [30] = "Jenny";
char S2 [7] = "khatri";
```

Strcat(S1, S2)



---

concatenation without Stocat

1) find Length of both strings.

```
char S1 [30] = "Jenny";
char S2 [7] = "khatri";

Int len1 = StrLen (S1);
Int len2 = Str Len (S2);
```

K h a t r i \0

↑ ↑ ↑ ↑ ↑ ↑ ↑

length of array who we will copy.

```
for (int i=0; i<=len2; i++)
    {
        S1 [len1+i] = S2 [i];
    }
    cout<< S1;
```

loops starts from 0 and goes all the way till 6. K n A t R i \0

0 1 2 3 4 5 6

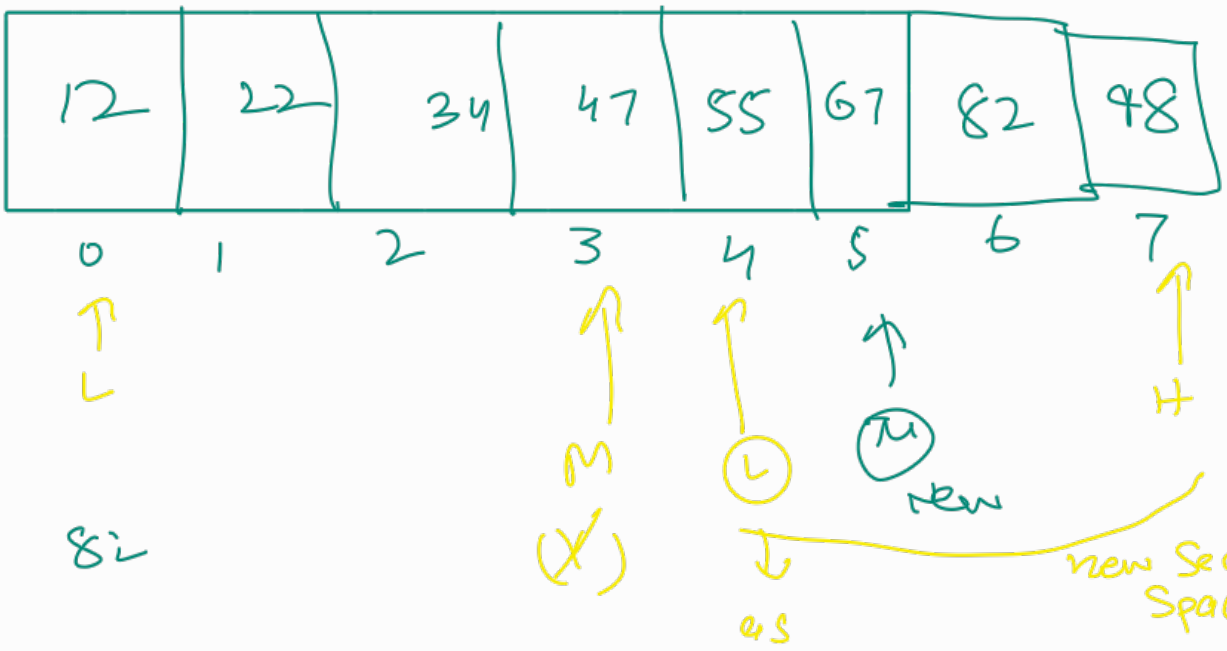
String comparision:-

Bill ⟷ Billy

S1 S2

Returns <0 because Bill is Less than Billy. the fifth character of Bill is '\0' is Less than fifth character of Billy that is 'y'.

Binary Search → must have a sorted array.



S.V is 82

$$\frac{\text{low index} + \text{high index}}{2}$$

```
int low=0;
int high= size-1;
 while( low<= high)
{int mid =  low + hight / 2;

  if (search value == array [mid ])
    {
       return mid;
    }
else if (search value > array [mid])
    {
       Low = mid+1;
    }
```

```
else if (searchValue < array[mid])
    {
        high = mid - 1;
    }
}
```

2D arrays.

Declaration: datatype arrayname [size1] [size2]

row size

column size.

e.g

float marks [2] [3]

rows \ columns	0	1	2
0	0,0	0,1	0,2
1	1,0	1,1	1,2

total elements we can store is 2x3 = 6

Initialising 2D array

float marks [2] [3] = {59.6, 12.3, 14.9, 31.7, 12.8, 91.0}

59.6	12.3	14.9
31.7	12.8	91.0

Another way

Int array [2] [3] = {{12, 13, 4}, {1, 2, 3}}

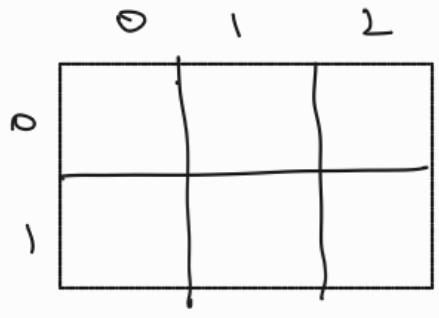
Row 1 Row 2

You can skip row size but not column size during initialisation.

Taking inputs from the user now. By using nested loops

1 Loop is for rows and 2 loop is for columns.

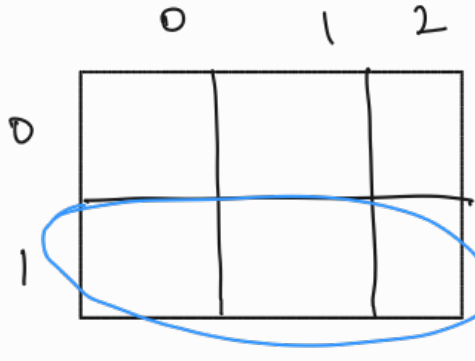
int arr [2] [3]




```
for (int i=0; i<=1; i++) → rows.
{
    for (int j=0; j<=2; j++) → columns
        cin>> arr [i] [j]
}
```

Accessing an element array name [rows] [columns]

Int array [2] [3];



we can process 2D arrays by:

lets say processing the Second Row.

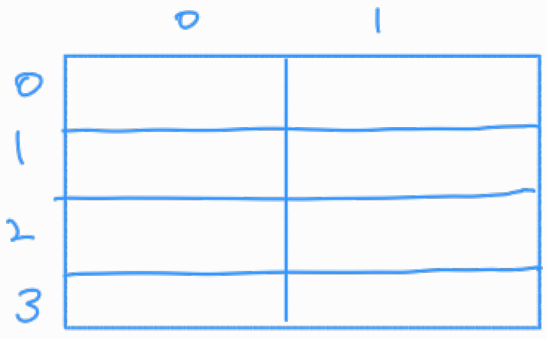
array [1] [0], array [1] [1], array [1] [2]

row = 1

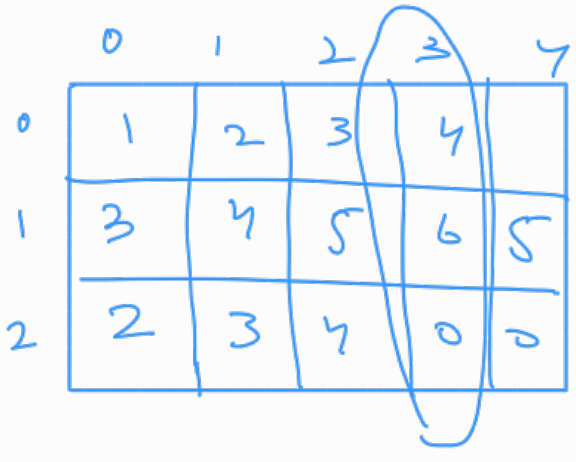
```
for (int i=0; i<no.of columns; i++)
    Process array[1][i];
```

example:- Take Inputs in Row 2 only.

Int array [4][2]

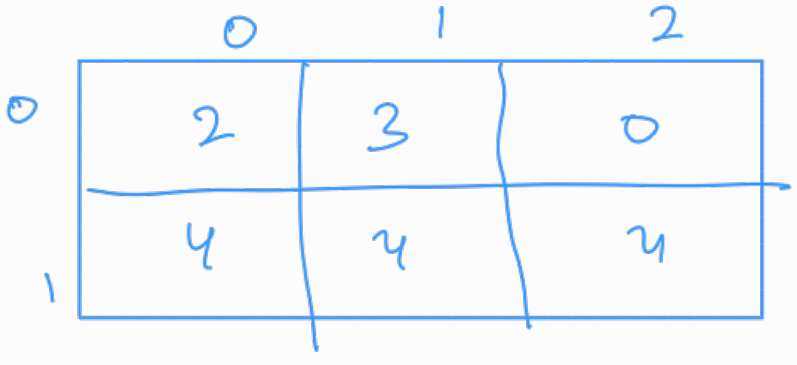



```
row = 2
for (int i=0; i<columns; i++)
    cin>> array[row][i];
```



4+6+0

Program to calculate the Sum of rows that is max.



→ outputs (12)

```
int array[2][3] = {{2,3,3}, {4,4,4}};
int max = array[0][0]   → initialise with first element
for (int i=0; i<2; i++)
{
    int sum=0;   → reset sum after calculating sum of one row.
    for (int j=0; j<3; j++)
    {
        sum = sum + array[i][j];
    }
    if (max < sum)
        max = sum;
}
cout << max;
```

Loop for rows

Loop for columns

Program for min row Sum:.

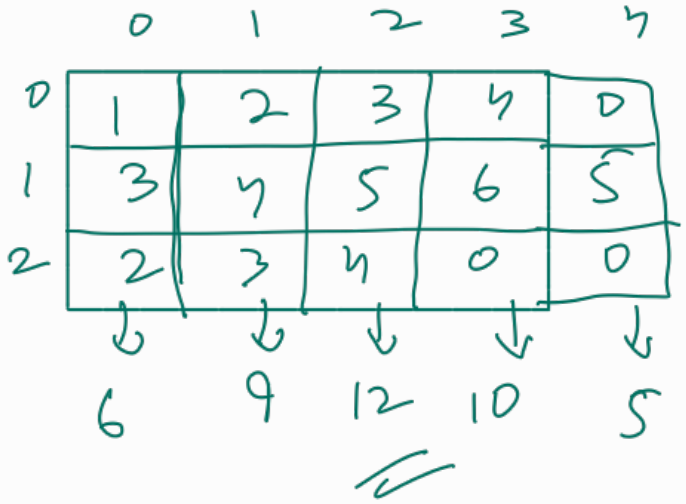
for this intialise min with the Sum of first row.

```
int min;
for (int i=0; i<3; i++)
    int sum=0
    for (int j=0; j<5; j++)
    {
        sum = sum + array[i][j];
        min = sum;
    }
```

```
if (sum < min)
    min = sum;

cout<<min;
```

Program for calculating sum for max coc.




```
int max = array[0][0];

    for (int i = 0; i<5; i++) → loop for columns
        int sum=0;
        for (int j = 0; j<3; j++) → loop for rows.
            sum = sum + array[i][j];

        if (sum > max)
            max= sum;

    cout << max;
```

Program to find max element in each column.

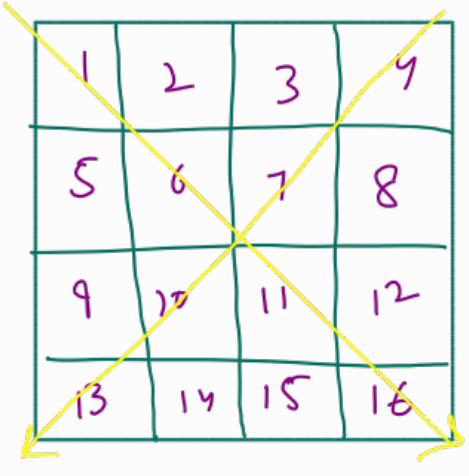
```
for (int i=0; i<col; i++)
{
    int max = array[0][col];   -> initial max with first element of col
    for (int j=0; j<row; j++)
    {
        if (max < array[j][i])
            max = array[j][i];
    }
    cout << max << endl;
```

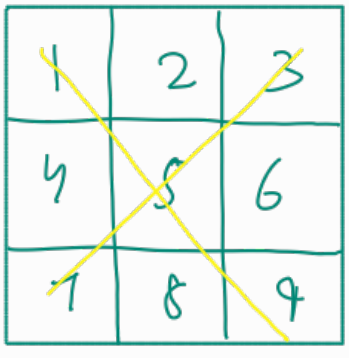
Diagonal Sum where the 2d array Rows = columns.
| a.ka Square matrix.

1 + 6 + 11 + 16 = 34

4 + 7 + 10 + 13 = 34

hence final Ans = 34 + 34 = 68





1 + 5 + 9 = 15

3 + 5 + 7 = 15

= 30 - 5 = 25

note: for odd Square matrix we cannot add 5 twice in Sum. hence we subtract one 5 from total

Because we only want to add each no once.

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

[0][0] + [1][1] + [2][2] + [3][3]

i=j

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

[0][3]

[1][2]

[2][1]

[3][0]

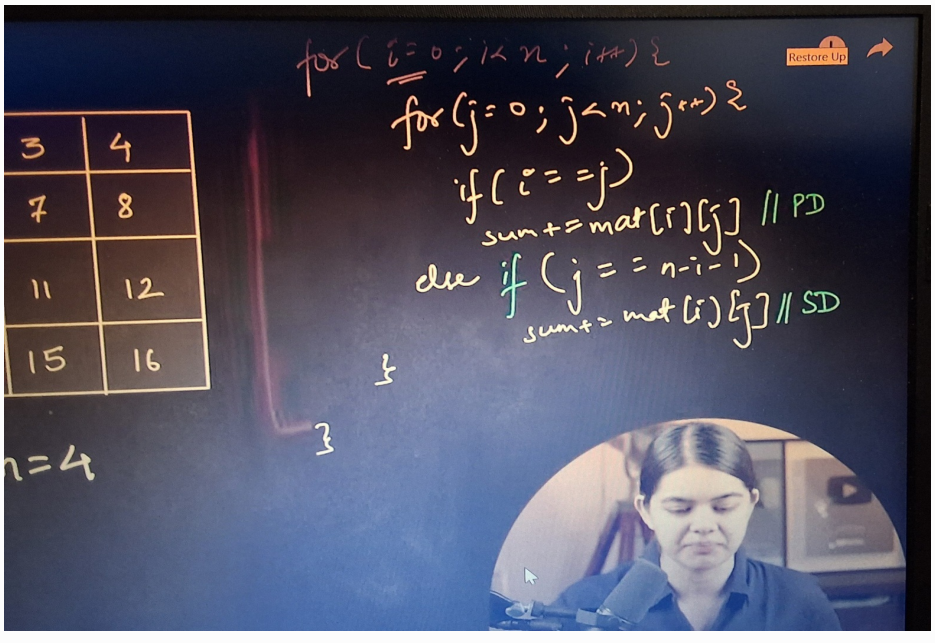
As row increases by 1 column decreases by 0.

j = n-1-i

n → no of rows / columns

column

rows



Matrix Multiplication for Matrix multiplication to be possible column of first matrix must be equal to the rows of second matrix.

2x3 3x2

Multiplication is possible

final Answer will be in (2x2)

$$a \rightarrow \begin{bmatrix} 0 & 5 & 2 \\ -1 & 1 & 0 \\ 7 & 4 & 3 \end{bmatrix} \quad b \rightarrow \begin{bmatrix} 1 & 2 \\ 0 & 4 \\ 3 & 5 \end{bmatrix}$$

3x3 3x2

$\Rightarrow$ $\begin{bmatrix} & \\ & \end{bmatrix}$ Answer 3x2

→ outer loop for rows (3)

inner loop for columns (2)

```
for (int i=0; i<3; i++)
    for (int j=0; j<2; j++)
        sum=0;
        for (int k=0; k< no. of columns of first Mat | no. of rows of second ; k++)
        {
```

Sum = Sum + a[i][k] * b[k][j]

Ans[i][j] = Sum; → for storing in Answer 2D array

Addition / Subtraction of two matrices. → dimensions of both matrices must be same

$$a\begin{bmatrix} 1 & 2 & 3 \\ 1 & 0 & 1 \end{bmatrix} + b\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

2×3 2×3

Resultant matrix $c\begin{bmatrix} 2 & 3 & 4 \\ 2 & 1 & 2 \end{bmatrix}$ 2×3

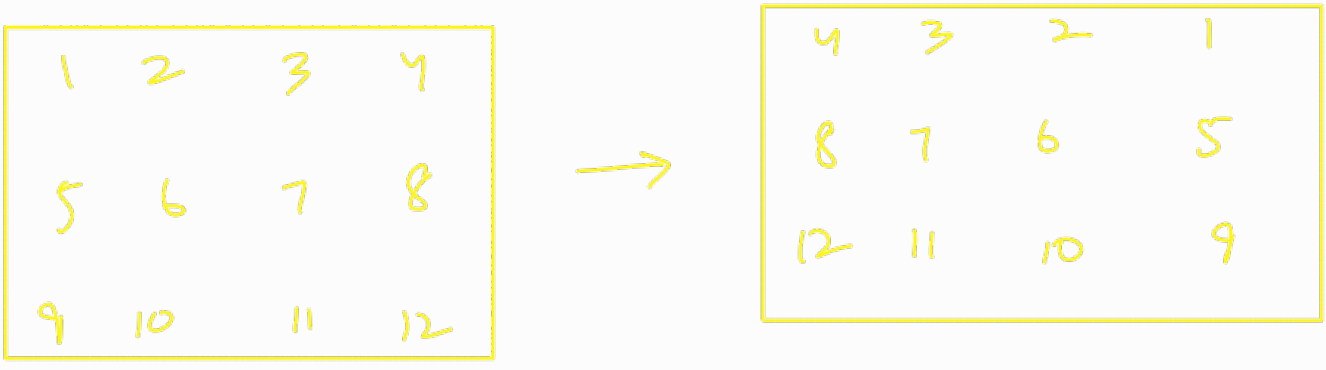
```
for (int i=0; i<2; i++)
{
    for (int j=0; j<3; j++)
    {
        c[i][j] = a[i][j] + b[i][j];
        cout<< c[i][j] << " ";
    }
    cout<<endl;
}
```

Passing 2D arrays in functions:.

(2D arrays are passed by reference)

→ you must specify the columns

Reversing 2D array



Rotation and Shifting of 1D arrays.

→ Rotation:- to right (1 step)

[1 2 3 4 5] → [5 1 2 3 4]



Rotation to left (1 step)

(1 2 3 4 5) → (2 3 4 5 1)



Shift Right (1 step)

[1 2 3 4 5] → (0 1 2 3 4]

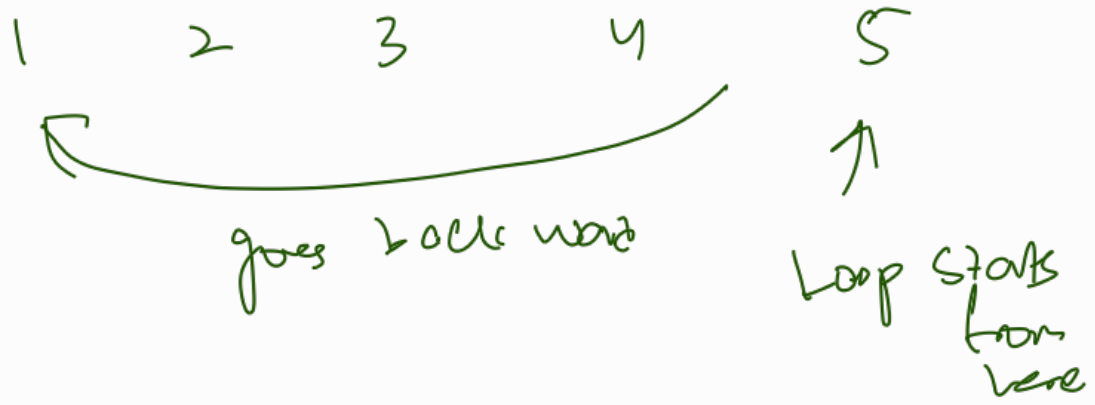
overwrites.

Shift Left (1 step)

[1 2 3 4 5] → [2 3 4 5 0)

Rotate a Array by 1:-

```
int temp = array [size-1];   → Last element
for (int i = (size-2); i>=0 ; i--)   → Start from second Last.
    array[i+1] = array[i];
array[0] = temp   → first element
```



To Rotate 'n' times to the Right
JUST put this function in a Loop.

e.g n=4

```
for (int i=0; i<4; i++)
    {
        Function to Rotate once
    }
```

Rotate a array to Left

3 4 8 1 7 6

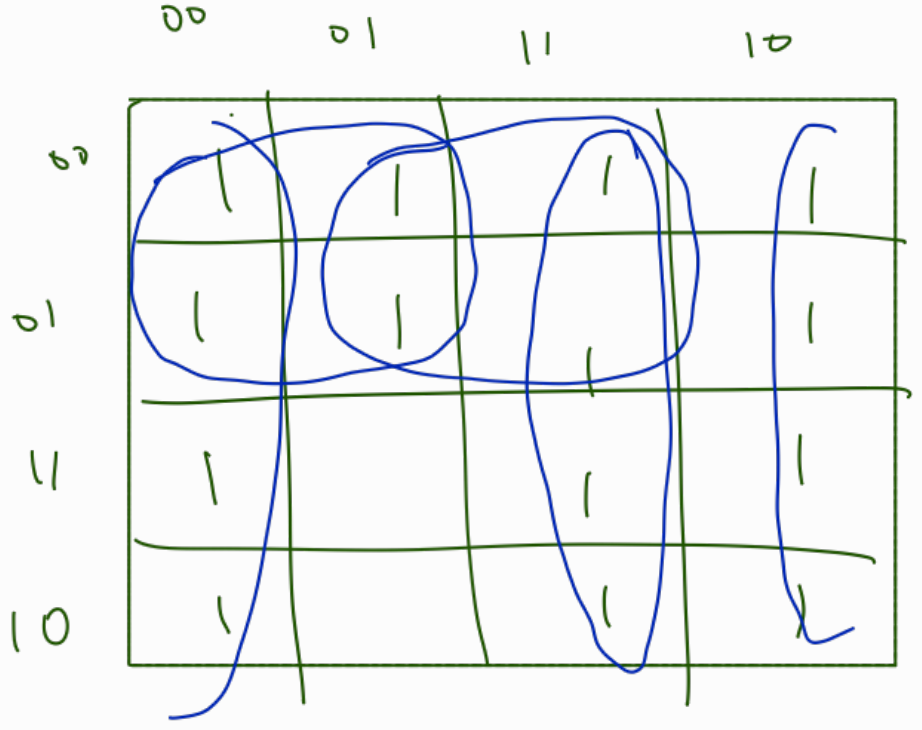


e.g

array = { 1 2 3 4 5 63

```
Int temp = array[0];
for (int i = 0; i < size-1; i++)
    arr[i] = arr[i+1];
arr[size-1] = temp;
```

Runs till second last



2D Character arrays

	0	1	2	3	4
0	a	b	c		
1	x	y	z		
2	p	q	r	s	t

char max [Rows] [columns];

↓

Intialisation =

char max [3] [6];

↙

6 because p q r s t contains '\0' at the end.

char max [] [] =

{ "abc", "pqrst", "xyz" };

↓ row1 ↓ row2 ↓ row3

Passing 2D arrays into a function:

note:

int a [] [3] = {0, 1, 2, 3, -4}

↙

0	1	2	3
-4	0	0	0

int a [] [3] = { {0, 1}, {2, -3} }

row1 ↗ row2 ↘

0	1	0
2	-3	0

Passing 2d arrays into functions;

```
void funname ( int arr[][] )        X
void funname ( int arr [ ][3] )     ✓
void funname ( int arr [2] [3] )    ✓
```

(M1)

```
const int r=3 , c=2;  → global constant

void name (int array [r] [c])
    {

        }

int main ()
    {
        name ( array);
    }
```

(M2)

```
const int c=3;

void name ( int arr [] [c], int r)
    {

        }

int main () {
    int arr [2] [3];
    name ( arr , 2 )

    }
```